Writing your First Program

MikroDuino Studio

Mikrotronics Pakistan

Writing Your First Program in MikroDuino Studio

Before you can write and upload programs, you need to create a project. A project contains all the source files, configuration settings, and build information required to compile your application for a specific microcontroller. In MikroDuino, creating a project is straightforward, allowing you to focus on writing code rather than managing complex build systems.

Creating a New Project

Start the MikroDuino IDE.

To create a new project:

1. Select **File → New Project**.
2. Enter a **Project Name**.
3. Choose the folder where the project will be stored.
4. Click **Create**.

The IDE will generate a new project containing all the required files.

The MikroDuino Project

Unlike Arduino where the entire application resides in single .ino file, the MikroDuino tends to keep your code organized into various folders and files. The project itself acts as a binder to keep all the project related files and settings. In previous chapter we have seen how you set the project properties of your application. Choosing the right microcontroller is important because the compiler will then select the appropriate hardware like RAM, Peripherals and EEPROM etc of the microcontroller.

The project has by default one folder src that contains all your source files. A single default file named main.cpp is always created, the file has main() function which is the starting point of your C++ program. You can write as much code in this file, but if you want to write specific drivers and want to keep code organized, just right click the src folder and add a new .cpp file all files that are present in the src folder are automatically compiled and included in the executable.

If your program requires to create header files, lie .h or .hpp (Mikroduino specific) Create a folder named includes under the project root and create include files there.

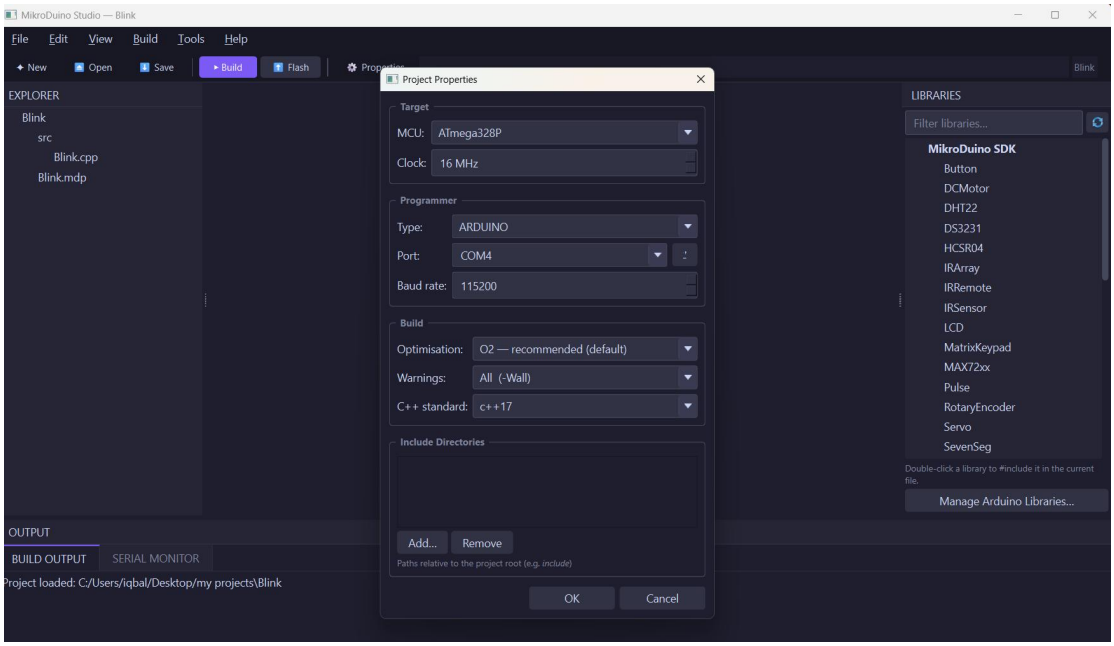
Why Select the Microcontroller Instead of the Board?

An Arduino Uno and many custom development boards use exactly the same ATmega328P microcontroller.

By selecting the MCU directly, the same application can later be used on:

- Arduino Uno
- Arduino Nano
- Custom PCB
- Breadboard prototype
- Industrial controller

without changing your source code. The compiler actually cares about microcontroller chosen instead of board as such.



README.md

In programming world we keep our notes in .md file instead of txt. It is just a text file but with specific order and format that many programmers prefer to keep their notes. A convenient place to document your project.

Typical information includes:

- project description
- hardware connections
- usage instructions
- revision history

Documentation becomes increasingly valuable as projects grow.

Writing Your First Program

Every C++ application begins with a function called `main()`.

A minimal MikroDuino application looks like this:

```
#include <mikroduino/mikroduino.hpp>

using namespace MikroDuino;

int main()
{
    while(true)
    {

    }
}
```

The `main()` function executes once after the microcontroller resets.

The infinite loop

```
while(true)
{
}
```

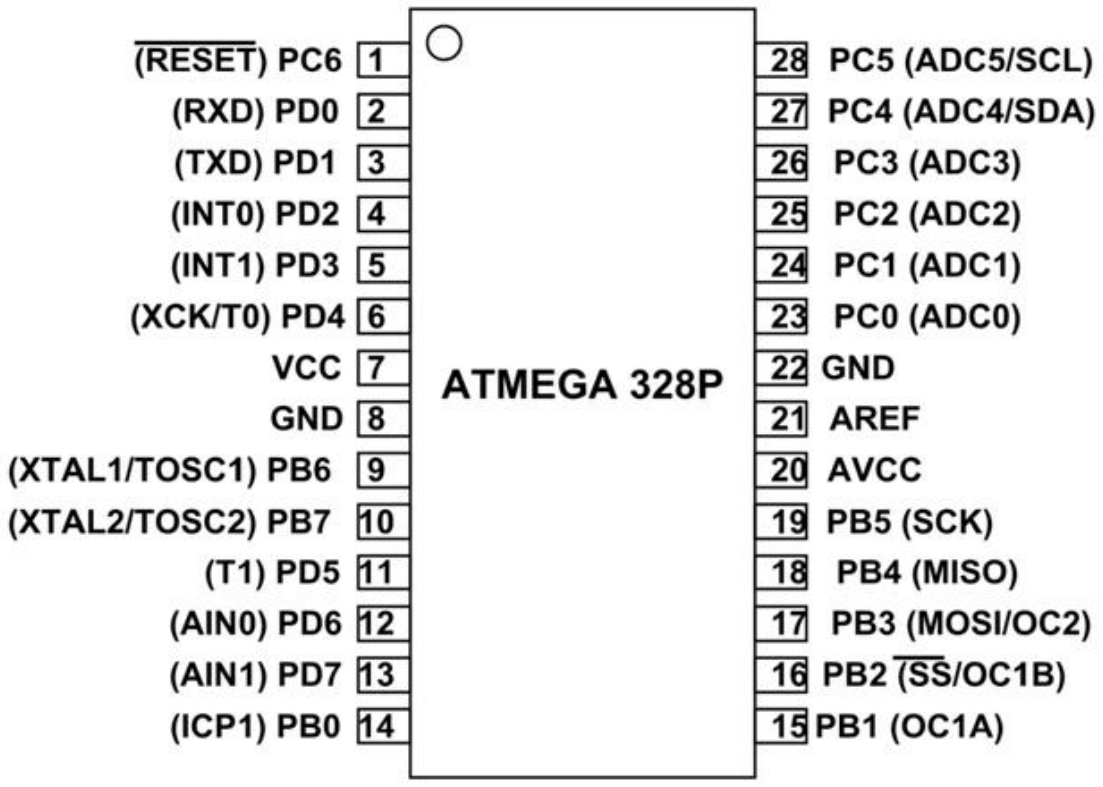
keeps the processor running forever.

Unlike desktop programs, embedded applications rarely terminate. Instead, they continuously monitor inputs, control outputs, and respond to events.

This is the loop() portion of Arduino programs. Any application wide settings, like configuration of pins, initialization of variables etc are done before this while(true) statement. That portion of code is run only once when the MCU is reset or starts execution.

First LED Blink Program

This usually the first program in any embedded systems, the idea is to turn a GPIO pin High and Low sequentially. The AVR microcontrollers organize the general



purpose input and output pins into PORTS, usually grouped in 8 pins. Not all microcontrollers have same number of PORTS and their pins (that is why we need to tell the compiler the microcontroller we are using). ATMega328P has many ports that we will discuss later, but here we will consider a port called PORTB. The PORTB has 6 pins named as PB0 to PB5 and C++ compiler recognizes these pins by these names. On an Arduino Nano and Arduino UNO a builtin testing LED is connected to PB5 pin. (In Arduino world this pin is called No. 13, Arduino defines the pin names as numerical numbers) This Arduino naming convention makes programming easy as by just changing the numerical value you are now addressing another pin, but this actually brings an extra unwanted code. MikroDuino does not use this convention (except in Arduino compatibility mode) rather it uses the standard naming convention used by microchip in the datasheet. So we will address this PORTB pin as PB5

```
/*
* Blink — MikroDuino GPIO example
* Toggles PB5 (Arduino Uno pin 13 / built-in LED) at 1 Hz
using the
* MikroDuino GPIO library.
* Target : ATmega328P @ 16 MHz
* Circuit: LED + 220 Ω resistor between PB5 and GND (or
use the built-in LED)
*/
#include <avr/io.h>
#include <util/delay.h>
#include <mikroduino/gpio.hpp>

using namespace MikroDuino;

static constexpr uint8_t LED = PB5;

int main() {
GPIO::output(LED);

while (true) {
GPIO::set(LED);
_delay_ms(200);
GPIO::clear(LED);
_delay_ms(200);
}
}
```

Lets dissect the program line by line to make sense of the code. After the initial comments about your program (which is a good convention) firs lines usually you write are the libraries that you need to use in the program. These are added to the program using #include <….> statements.

```
#include <avr/io.h>
#include <util/delay.h>
#include <mikroduino/gpio.hpp>
```

The first two statements #include <avr.io.h> and util/delay.h are the standard C++ libraries provided by AVR-GCC compiler. These two libraries provide the mapping for our GPIO pins and define a function that we will use to halt the program for a certain period of time.

```
The third library #include <mikroduino/gpio.hpp>
```

Is a library provided by MikroDuino therefore it has the folder name mikroduino attached with it. This file contains all the necessary code to manipulate the GPIO pins (similar to arduino pinMode, digitaRead etc). We will see in a while how to access the member functions of this library.

```
using namespace MikroDuino;
```

All libraries developed by MikroDuino are organized under this namespace, so that if we have two libraries with the same name, we can differentiate between them using their namespace. One namespace however cannot hold two files with the same name. So when accessing these functions, we have to write the namespace as part of the function name. This is usually best practice, but if we don't expect a conflict, we publicly say using namespace MikroDuino, then just need to give the name of function, it will automatically be looked in this namespace.

```
static constexpr uint8_t LED = PB5;
```

This line declares a constant named `LED` whose value is the pin number `PB5`.

```
static constexpr uint8_t LED = PB5;
```

Let's understand each part.

`uint8_t`

`uint8_t` is an unsigned 8-bit integer type defined in `<stdint.h>`.

It can store values from **0 to 255**. The standard C used to have data types like int, char etc. On different compilers int might mean different data type, like in embedded systems it might mean a 16 bit number, while on laptop it means 32 bit number. To make it clear and reduce ambiguity C++ now introduces data types which clearly define the size of variable created.

Since AVR microcontrollers use 8-bit registers and GPIO pin numbers are small integers, `uint8_t` is the ideal data type.

`constexpr`

The keyword `constexpr` tells the compiler that the value is **known at compile time** and will never change during program execution. In otherwords it's a constant value.

```
constexpr uint8_t LED = PB5;
```

This has several advantages:

- ✓ The value is treated as a constant.
- ✓ It can be used wherever a compile-time constant is required.
- ✓ The compiler substitutes the actual value directly into the generated machine code.
- ✓ No RAM is allocated for the variable in most cases.

For example:

```
constexpr uint8_t LED = PB5;

pinMode(LED, OUTPUT);
digitalWrite(LED, HIGH);
```

The compiler effectively replaces `LED` with the value of `PB5` before generating the executable.

`constexpr` VS `const`

Many beginners and C programmers are familiar with `const`.

```
const uint8_t LED = PB5;
```

For simple variables like this, `const` and `constexpr` often produce identical machine code. However, `constexpr` is stronger because it **guarantees** that the value is available at compile time. For this reason, modern C++ code generally prefers `constexpr` for fixed constants.

`static`

The meaning of `static` depends on where it is used.

In this example, assume the declaration is at **file scope**, outside any function:

```
static constexpr uint8_t LED = PB5;
```

Here, `static` gives the variable **internal linkage**.

This means:

- ✓ The constant is visible only within the current source file.
- ✓ Other source files cannot access it.
- ✓ It helps prevent name conflicts in large projects.

Suppose you have two files:

main.cpp

```
static constexpr uint8_t LED = PB5;
```

test.cpp

```
static constexpr uint8_t LED = PD2;
```

Both files can have a variable named `LED` without conflicting because each one is private to its own source file.

Without `static`, a global variable has external linkage by default, meaning other source files could potentially refer to it (unless it is otherwise declared in a way that gives it internal linkage). Using `static` makes the intent explicit: this constant belongs only to this source file.

Why Use Both?

Writing

```
static constexpr uint8_t LED = PB5;
```

expresses two important ideas:

- `constexpr` → the value is a compile-time constant.
- `static` → the constant is private to this source file.

This combination is common in embedded programming because it produces efficient code while keeping project organization clean.

Summary

Keyword	Purpose
uint8_t	Stores an unsigned 8-bit integer (0 – 255).
constexpr	Declares a compile-time constant that cannot change.
static	Limits the constant's visibility to the current source file (when used at file scope).

For small embedded applications, you may also see:

```
constexpr uint8_t LED = PB5;
```

or even

```
const uint8_t LED = PB5;
```

All three are valid, but `static constexpr` is a clear, modern C++ style for defining file-local constants in embedded software.

```
GPIO::output(LED); // Set pin mode
```

This statement looks different to most Arduino programmers. This is MikroDuino equivalent to pinMode(). The GPIO is a class or object type declared in MikroDuino/gpio.hpp library file. Usually when we use objects, we create an instance of the object, in that case every instance has its own set of functions. Since the functions in gpio class are all going to act on common AVR registers, we have declared this class as static. Which means you don't need to create its instance and it can be used directly.

Output is a function in this class, so when accessing a function or member of a static class object, you use :: as connector between class name and member name. So GPIO::output(LED) actually says that execute the output function of GPIO static class object and use LED (PB5) as an argument. This statement therefore sets the PB5 pin to which LED is connected as output.

```
GPIO::set(LED);
_delay_ms(200);
GPIO::clear(LED);
_delay_ms(200);
```

Similarly GPIO::set(LED) function sets the pin defined in LED (PB5) High. This statement is MikroDuino version of digitalWrite(). In order to set the pin Low, we have another function GPIO::clear().

_delay_ms(200) is similar to Arduino delay() function. It blocks the program execution for 200 milli seconds. This function is defined in the compiler's own libraries named util/delay.h Since many microcontrollers can run at varying crystal speeds, the delay is calculated during compilation to effectively produce exact timing.

The Arduino delay() function is not very accurate, It uses a timer register overflow, which overflows at value greater than exact 1000 resulting in slightly inaccurate delay. This does not matter much in common usage, but where you need precision, it must be accurate.

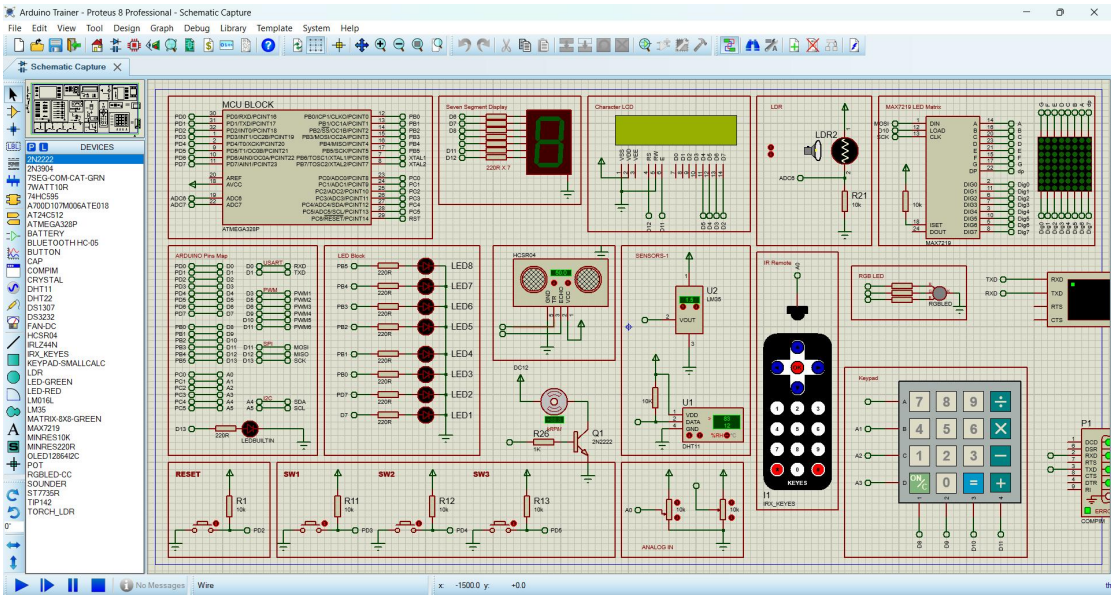
So you write your application in your project. Once you are done editing just press build and it will invoke the AVR-GCC compiler and its linker. All libraries are compiled again to match your microcontroller (only the ones that you are using). After the compilation is successful press flash button and your code will be transferred to your Arduino board.

In case you are not using Arduino compatible board, then you will need a separate standalone programmer like USBASP connect its output pins to programming specific pins of your microcontroller and flash using it.

Proteus Simulator

Labcenter electronics have made a very good software, Proteus. It is basically meant for schematic drawing and PCB designing, it however contains vsm models of many components, so that you can experience simulated electronics. Fortunately it has a bunch of microcontrollers as well, like ATMega328P and many others. You can set the properties of the microcontroller like clock speed, fuses etc as well as mention

the .hex file that you want to execute. I can say with confidence that almost 90-95% of your code runs on it.



I frequently test my applications on it. When you build the application you will find a build folder in the project folder. It contains the .hex file.